

2026 CCF 非专业级别软件能力认证第一轮

(CSP-S1) 提高级 C++ 语言试题

认证时间：2026 年 9 月 19 日 14:30~16:30

考生注意事项：

- 试题共 11 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典、电子手表等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 执行下列代码后，cnt 的值是（ ）

```
int x = 2026, cnt = 0;
while (x) {
    x &= x - 1;
    cnt++;
}
```

A. 6 B. 7 C. 11 D. 8

2. 用权值 {1, 2, 3, 4, 5, 6, 7, 8} 构造哈夫曼树，其带权路径长度是（ ）

A. 108 B. 96 C. 99 D. 102

3. 把 1 到 1000 的所有整数按十进制写出，数字“1”总共出现了多少次（ ）

A. 300 B. 271 C. 301 D. 320

4. 将 5 封信随机装入 5 个写好地址的信封（每封一个），恰好有 2 封装对的方案数是（ ）

A. 44 B. 24 C. 10 D. 20

5. $3^{2026} \bmod 100$ 的值是（ ）

A. 29 B. 9 C. 43 D. 81

6. 有 5 堆石子排成一行，重量依次为 4, 1, 3, 2, 5。每次只能把相邻的两堆合并成一堆，代价为这两堆重量之和。将所有石子合并成一堆的最小总代价是（ ）

A. 36 B. 35 C. 34 D. 33

7. 树状数组维护长度 $n = 16$ 的序列。查询前缀和 $\text{sum}(11)$ 与单点修改 $\text{add}(3, x)$ 分别需要访问树状数组中多少个下标（ ）

A. 3 和 4 B. 4 和 4 C. 3 和 5 D. 4 和 3

8. 有向无环图 G 顶点集为 $\{1, 2, 3, 4\}$, 边集为 $\{(1, 2), (1, 3)\}$, 顶点 4 与任何顶点均不相邻。该图不同的拓扑序共有多少种 ()
- A. 12 B. 8 C. 4 D. 6
9. 某分治算法满足 $T(n) = T(n/3) + T(2n/3) + \Theta(n)$, $T(1) = O(1)$, 则 $T(n)$ 是 ()
- A. $\Theta(n \log n)$ B. $\Theta(n^2)$ C. $\Theta(n^{1.5})$ D. $\Theta(n)$
10. 无根树含 9 个结点 (编号为 1—9), 边集为 $\{(1, 2), (1, 3), (2, 4), (2, 5), (3, 6), (6, 7), (7, 8), (5, 9)\}$ 。该树的直径 (以边数计) 与重心分别是 ()
- A. 直径 6, 重心为结点 3 B. 直径 7, 重心为结点 2
C. 直径 8, 重心为结点 1 D. 直径 7, 重心为结点 1
11. 一张有向图缩点后得到的有向无环图含 6 个顶点, 其中入度为 0 的顶点有 3 个、出度为 0 的顶点有 4 个。为使原图变成强连通图, 至少需要添加多少条有向边 ()
- A. 7 B. 6 C. 4 D. 3
12. 含 6 个结点的不同形态的二叉树共有多少棵 (结点不带标号, 区分左右子树) ()
- A. 42 B. 429 C. 132 D. 720
13. 字符串 $S = \text{"ababaabab"}$, 其所有既是真前缀又是真后缀的子串 (非空) 的长度之和是 ()
- A. 4 B. 6 C. 7 D. 5

14. 用归并排序统计逆序对, 合并部分的核心代码为

```
// 归并 a[l..mid] 与 a[mid+1..r], 同时累加逆序对
if (a[i] <= a[j]) {
    tmp[k++] = a[i++]; // 取左半段元素
}
else {
    tmp[k++] = a[j++]; // 取右半段元素
    ans += mid - i + 1;
}
```

若把判断条件中的 $a[i] \leq a[j]$ 改成 $a[i] < a[j]$, 则 ans 统计出的结果是 ()

- A. 完全不变
B. 变为原来的两倍
C. 变为满足 $i < j$ 且 $a[i] \geq a[j]$ 的数对个数
D. 变为原来的一半

15. 执行 `power(2, 100, 1000)` 调用下列函数, 返回值是 ()

```
long long power(long long a, long long b, long long p) {
    long long r = 1 % p;
    while (b) {
        if (b & 1)
            r = r * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return r;
}
```

A. 576

B. 376

C. 976

D. 176

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题 1.5 分, 选择题 3 分, 共计 40 分)

(1)

```
01 #include <iostream>
02 #include <string>
03 using namespace std;
04 int a[100];
05 string s;
06 int gen[13] = {1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1};
07 int main() {
08     cin >> s;
09     for (int i = 0; i < 32; ++i) {
10         a[i] = s[i] - '0';
11     }
12     for (int i = 32; i < 44; ++i) {
13         a[i] = 0;
14     }
15     for (int i = 0; i < 32; ++i) {
16         if (a[i] == 0) continue;
17         for (int j = 0; j < 13; ++j) {
18             a[i + j] ^= gen[j];
19         }
20     }
21     for (int i = 32; i < 44; ++i) {
22         cout << a[i];
23     }
24     cout << endl;
25     return 0;
26 }
```

(说明：输入保证为一个长度恰为 32 的 '0'/'1' 字符串。)

判断题

16. (1 分) 当输入为 32 个 '0' 时，程序输出 12 个 0。()
17. 程序运行结束后，数组 a 中下标从 0 到 31 的元素一定全部为 0。()
18. 若将第 12~14 行 (为 a[32] 到 a[43] 补 0 的循环) 删除，会改变程序输出结果。()

单选题

19. 关于第 6 行定义的数组 gen，下列说法正确的是 ()。
- A. gen 共有 12 个元素，表示一个 12 位的除数
 - B. gen 共有 13 个元素，表示一个 13 位的被除数
 - C. gen 共有 13 个元素，其中 gen[0] 是除数的最高位
 - D. gen 共有 13 个元素，其中 gen[12] 是除数的最高位
20. 该程序实现的功能，最准确的说法是 ()。
- A. 将输入的 32 位串看成二进制数 M ，输出 M 与 13 位二进制数 1100000001111 按位异或的结果
 - B. 将输入串视为 32 位二进制数 M ，在其后补 12 个 0 (即计算 $M \times 2^{12}$)，再对它用 1100000001111 作模 2 除法求余数，并输出 12 位余数
 - C. 对输入的 32 位串逐位取反并输出结果
 - D. 统计输入串中 1 的个数，并把该个数用 12 位二进制表示后输出
21. 若将第 16 行 if (a[i] == 0) continue; 删除，说法正确的是 ()。
- A. 程序输出的结果不会改变
 - B. 可能造成程序运行错误
 - C. 程序能够正常输出一个 12 位 '0'/'1' 串，但是输出结果与输入的 s 无关
 - D. 程序运行结束后，a[0] 的值一定为 0

(2)

```
01 #include <iostream>
02 using namespace std;
03 int n, m, a[100007], L, R, lg[100007], i, j, t, dp[100007][25], pw[25];
04 int gcd(int x, int y) {
05     if (y == 0) return x;
06     return gcd(y, x % y);
07 }
08 int main() {
09     cin >> n >> m;
10     for (i = 1; i <= n; i++) cin >> a[i];
11     t = 0;
12     pw[0] = 1;
```

```

13     for (i = 1; i <= 24; i++) pw[i] = pw[i - 1] * 2;
14     for (i = 1; i <= 100000; i++)
15         if (pw[t + 1] > i) lg[i] = t;
16         else t++, lg[i] = t;
17     for (i = 1; i <= n; i++)
18         dp[i][0] = a[i];
19     for (j = 1; j <= lg[n]; j++)
20         for (i = 1; i + pw[j] - 1 <= n; i++) {
21             dp[i][j] = gcd(dp[i][j - 1], dp[i + pw[j] - 1][j - 1]);
22         }
23     for (i = 1; i <= m; i++) {
24         cin >> L >> R;
25         cout << gcd(dp[L][lg[R-L+1]], dp[R-pw[lg[R-L+1]]+1][lg[R-L+1]]) << endl;
26     }
27     return 0;
28 }

```

(说明：保证 $1 \leq n \leq 100000$ ，每次查询满足 $1 \leq L \leq R \leq n$ ，且数组 a 的元素均为正整数。)

判断题

22. 当 $n = 5$ ， $a = \{4, 2, 6, 3, 9\}$ ，且仅有一次查询 $L = 2$ 、 $R = 5$ 时，输出为 1。()
23. 当某次查询的区间长度为 1（即 $L = R$ ）时，这次查询的输出一定等于 $a[L]$ 。()
24. 任意一次查询的输出结果一定不小于该查询区间内的最小值。()

单选题

25. 对于 $j \geq 1$ ，数组 $dp[i][j]$ 保存的是 ()。
- A. 从 $a[i]$ 开始连续 j 个数的最大公约数
- B. 从 $a[i]$ 开始连续 2^j 个数的最大公约数
- C. $a[i]$ 与 $a[j]$ 的最大公约数
- D. 从 $a[1]$ 到 $a[i]$ 的最大公约数
26. 若把一次求最大公约数的运算视为 $O(1)$ ，则第 17~22 行建表过程的时间复杂度为 ()。
- A. $\Theta(n)$ B. $\Theta(n \log n)$ C. $\Theta(n^2)$ D. $\Theta(mn)$
27. 设 x 为一次查询的区间长度（即 $x = R - L + 1$ ），则使得 $lg[x]=5$ 的 x 的取值范围是 ()。
- A. $[16, 31]$ B. $[17, 32]$ C. $[32, 63]$ D. $[33, 64]$

(3)

```
01 #include <iostream>
```

```

02 using namespace std;
03 int n,fa[100007],f[100007],ans;
04 int main() {
05     cin >> n;
06     for (int i = 2; i <= n; ++i) {
07         cin >> fa[i];
08     }
09     for (int i = n; i >= 2; --i) {
10         if (f[fa[i]] + f[i] + 1 > ans) {
11             ans = f[fa[i]] + f[i] + 1;
12         }
13         if (f[i] + 1 > f[fa[i]]) {
14             f[fa[i]] = f[i] + 1;
15         }
16     }
17     cout << ans << endl;
18     return 0;
19 }

```

(说明：输入第一行为结点个数 n ，第二行为 $n - 1$ 个整数，依次表示结点 $2 \sim n$ 的父结点编号，满足 $2 \leq n \leq 100000$ 且 $1 \leq fa[i] < i$ ，根结点为 1。)

判断题

28. 当 $n = 5$ ， $fa[2] \sim fa[5] = \{1, 2, 3, 4\}$ 时，程序输出 4。()
29. 程序输出前， $f[1]$ 的值一定等于 ans 的值。()
30. 将第 10~12 行与第 13~15 行两个 `if` 语句的顺序交换后，程序的输出结果不受影响。()

单选题

31. 程序输出的 ans 表示的是 ()。
- A. 树中距离最远的两个结点之间路径所经过的边数
 - B. 根结点 1 到最远叶子结点之间路径所经过的边数
 - C. 树中叶子结点的个数
 - D. 所有结点的父结点编号之和
32. 当 $n = 7$ ， $fa[2] \sim fa[7] = \{1, 1, 2, 2, 3, 3\}$ 时，输出为 ()。
- A. 2 B. 3 C. 4 D. 5
33. 当 $n = 10$ ，满足输出为 9 的合法输入种类数为 ()。
- A. 0 B. 9 C. 256 D. 512

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (平衡路线) 给定一张有 n 个顶点、 m 条边的无向图，每条边带有符号 $+$ 或 $-$ 。对于一条从顶点 s 到顶点 t 的路线，允许重复经过顶点和边，定义一条路线的权值如下：记 n^+ 、 n^- 分别为经过的 $+$ 边数和经过的 $-$ 边数，则该路线的权值为 $|n^+ - n^-|$ 。

请计算从 s 到 t 的路线的最小权值。若不存在从 s 到 t 的路线，则输出 -1 。

输入第一行为四个整数 n, m, s, t 。接下来 m 行，每行给出两个整数 a, b 和一个字符 $+$ 或 $-$ ，描述一条连接 a 与 b 的无向边及其符号。

数据满足 $2 \leq n \leq 2 \times 10^5$ ， $1 \leq m \leq 4 \times 10^5$ ， $1 \leq s, t \leq n$ 且 $s \neq t$ ， $1 \leq a, b \leq n$ ，可能出现重边。

以下程序通过 BFS 求出最小权值。请补全程序。

```
01 #include <iostream>
02
03 constexpr int N = 200005;
04 constexpr int M = 400005;
05
06 int n, m, s, t;
07 int h[N], e[M << 1], ne[M << 1], w[M << 1], idx;
08 int q[N], d[N], c[N];
09
10 void add(int a, int b, int z) {
11     e[idx] = b;
12     w[idx] = z;
13     ne[idx] = h[a];
14     h[a] = idx++;
15 }
16
17 int main() {
18     std::cin >> n >> m >> s >> t;
19     for (int i = 1; i <= n; i++)
20         h[i] = d[i] = c[i] = -1;
21     for (int i = 0; i < m; i++) {
22         int a, b;
23         char op[2];
24         std::cin >> a >> b >> op;
25         int z = ①;
26         add(a, b, z);
27         add(b, a, z);
28     }
29     int hh = 0, tt = 0;
30     int p = 0, ng = 0, ok = 1;
31     q[tt++] = s;
32     d[s] = c[s] = 0;
```

```

33  while ( ② ) {
34      int x = q[hh++];
35      for (int i = h[x]; i != -1; i = ne[i]) {
36          int y = e[i];
37          if (w[i] > 0) p = 1;
38          if (w[i] < 0) ng = 1;
39          if (d[y] == -1) {
40              d[y] = ③;
41              c[y] = c[x] ^ 1;
42              q[tt++] = y;
43          } else if ( ④ )
44              ok = 0;
45      }
46  }
47  if (d[t] == -1) {
48      std::cout << -1;
49      return 0;
50  }
51  if (!p || !ng) {
52      std::cout << d[t];
53      return 0;
54  }
55  if ( ⑤ ) std::cout << 0;
56  else std::cout << 1;
57  return 0;
58 }

```

34. ①处应填 ()

- A. `op[0] == '+' ? 0 : 1`
 C. `op[0] == '+' ? 1 : -1`

- B. `op[0] == '+'`
 D. `op[0] == '-' ? 1 : 0`

35. ②处应填 ()

- A. `hh < n` B. `tt < n` C. `hh <= tt` D. `hh < tt`

36. ③处应填 ()

- A. `d[y] + 1` B. `d[x] + 1` C. `d[x]` D. `d[x] - 1`

37. ④处应填 ()

- A. `c[y] == c[x]` B. `w[i] == 1`
 C. `c[y] != c[x]` D. `d[y] + 1 != d[x]`

38. ⑤处应填 ()

- A. `ok && c[s] == c[t]` B. `ok && c[s] != c[t]`
 C. `!ok || c[s] == c[t]` D. `!ok && c[s] != c[t]`

(2) (标准答案) 有 n 名学生参加一次考试, 考试共有 m 道选择题, 每道题只有 A、B 两个选项。

第 i 名学生的作答用一个长度为 m 的字符串 a_i 表示。若最终公布的标准答案与该学生在某道题上的作答相同, 则该学生得 1 分, 否则不得分。记第 i 名学生最终得到的总分为 r_i 。

给定每名学生的目标分数 x_i 。现在需要构造一份标准答案, 使 $\sum_{i=1}^n |r_i - x_i|$ 尽可能大。

数据满足 $1 \leq n \leq 20$, $1 \leq m \leq 300$, $0 \leq x_i \leq m$ 。

以下程序从枚举符号的角度处理 $\sum_{i=1}^n |r_i - x_i|$, 把它写成更易优化的形式。

对于非零整数 x , `__builtin_ctzll(x)` 返回 x 的二进制表示末尾连续 0 的个数。

`__builtin_popcountll(x)` 返回 x 的二进制表示中 1 的个数。

程序输出一组满足要求的标准答案。请补全程序。

```
01 #include <cstdlib>
02 #include <iostream>
03 #include <string>
04 #include <vector>
05
06 using namespace std;
07
08 typedef long long ll;
09 typedef unsigned long long ull;
10
11 int main() {
12     int n, m;
13     cin >> n >> m;
14     vector<ll> x(n), c(n);
15     for (int i = 0; i < n; i++) {
16         cin >> x[i];
17         c[i] = ①;
18     }
19     vector<string> a(n);
20     for (int i = 0; i < n; i++)
21         cin >> a[i];
22
23     vector<int> s(n, -1);
24     vector<ll> q(m, 0);
25     ll C = 0, S = 0;
26     for (int i = 0; i < n; i++) {
27         C -= c[i];
28         for (int j = 0; j < m; j++) {
29             if (a[i][j] == 'A') q[j]--;
30             else q[j]++;
31         }
32     }
33     for (int j = 0; j < m; j++) S += abs(q[j]);
```

```

34  ll ans = C + S;
35  ull best = 0, lst = 0;
36
37  for (ull mask = 1; mask < (1ULL << n); mask++) {
38      ull g = ②;
39      ull d = g ^ lst;
40      int k = ③;
41      C -= ④;
42      for (int j = 0; j < m; j++) {
43          ll old = q[j];
44          int v = (a[k][j] == 'A' ? 1 : -1);
45          q[j] -= 2ll * s[k] * v;
46          S += abs(q[j]) - abs(old);
47      }
48      s[k] = -s[k];
49      if (C + S > ans) {
50          ans = C + S;
51          best = g;
52      }
53      lst = g;
54  }
55
56  for (int i = 0; i < n; i++) {
57      if (best >> i & 1) s[i] = 1;
58      else s[i] = -1;
59  }
60
61  string res(m, 'A');
62  for (int j = 0; j < m; j++) {
63      ll v = 0;
64      for (int i = 0; i < n; i++) {
65          if (a[i][j] == 'A') v += s[i];
66          else v -= s[i];
67      }
68      if (⑤) res[j] = 'A';
69      else res[j] = 'B';
70  }
71  cout << res << endl;
72  return 0;
73 }

```

39. ①处应填 ()

A. $2 * x[i] - m$

B. $-m + 2 * x[i] + 1$

C. $m - 2 * x[i]$

D. $m + 2 * x[i]$

40. ②处应填 ()

A. `mask | (mask >> 1)`

C. `mask & (mask >> 1)`

B. `mask ^ (mask >> 1)`

D. `mask ^ ((mask >> 1) + 1)`

41. ③处应填 ()

A. `__builtin_ctzll(d) + 1`

C. `__builtin_ctzll(g)`

B. `__builtin_popcountll(d)`

D. `__builtin_ctzll(d)`

42. ④处应填 ()

A. `2ll * s[k] * c[k]`

C. `2ll * (s[k] - c[k])`

B. `s[k] * c[k]`

D. `2ll * c[k]`

43. ⑤处应填 ()

A. `v >= (n & 1)`

C. `v + (n & 1) >= 0`

B. `v > (n & 1)`

D. `v * (n & 1) >= 0`